SUPER MARIO ODYSSEY



깃허브

| https://github.com/Durumyisking/Dx3dEngine

게임 제목 및 장르

| Super Mario Odyssey

| 플랫포머, 액션 어드벤처

개발 인원

| 5인 제작

개발 기간

| 2023년 6월 ~ 11월 (6개월)

개발 환경

| Window, C++

개발 도구

| VS 2022, DirectX 11, Window API, Git, HLSL 5.0

| PhysX 4.1.2, Assimp, FMOD, IMGUI, RenderDoc

SUPER MARIO ODYSSEY

PBR (Physically Based Rendering)

- 사실적인 표현을 위한 물리 기반 렌더링
  -> 빛을 물리 법칙대로 계산하는 렌더링 파이프라인 구현

- IBL(Image Based Light)을 사용한 전역 조명 구현

- 다양한 라이트 구현
  (directional,point,spot light)

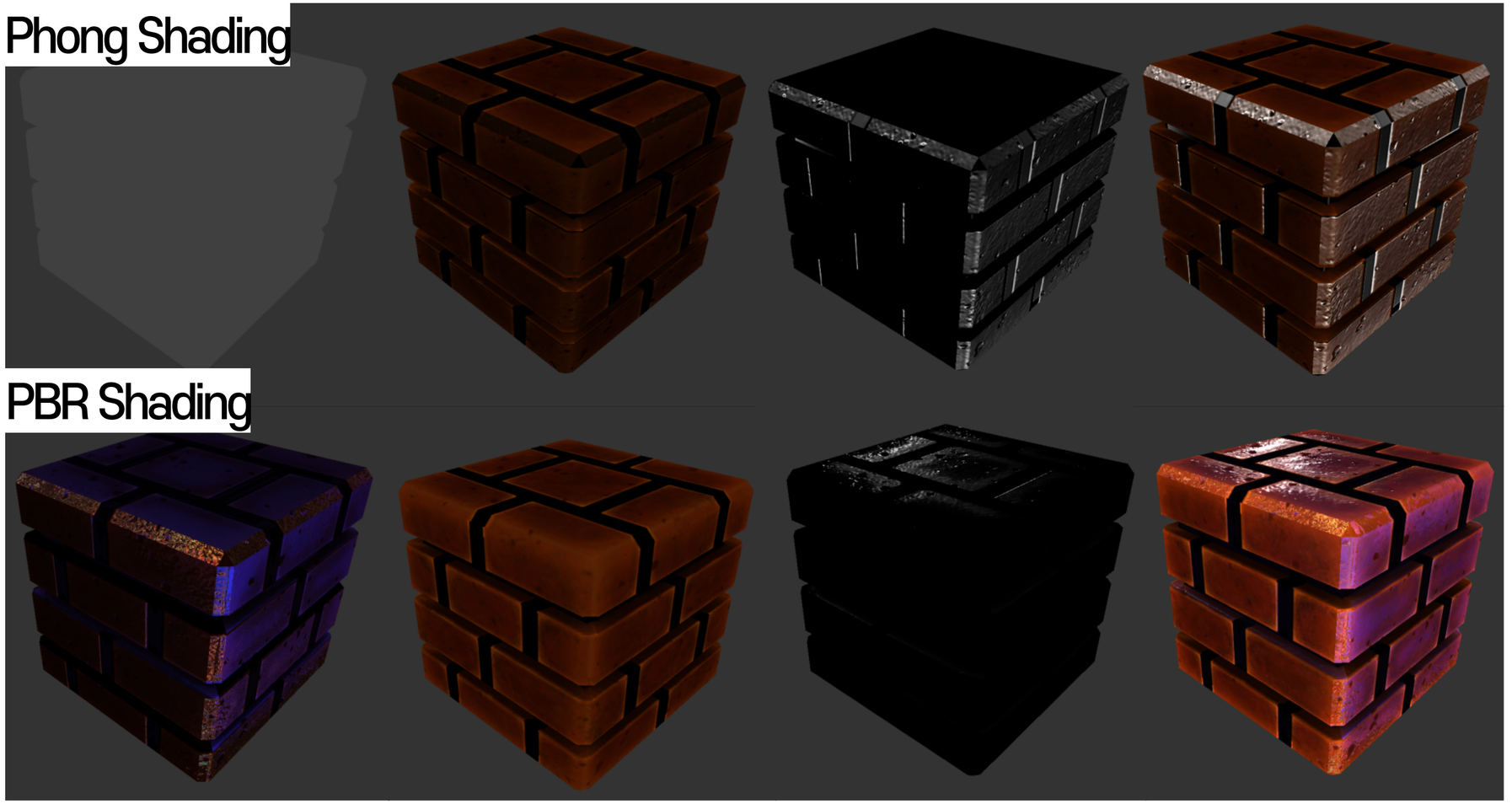
- 사실적인 반사 구현을 위한 프레넬 구현



SUPER MARIO ODYSSEY

- PBR (Physical based Rendering)
- Depth Map그림자 & PCF
- Deferred Rendering
- 분리된 모델 본 연결
- Instancing
- Frustum Culling
- FSM & Capture기능
- 애니메이션 이벤트

PBR (Physical based Rendering)

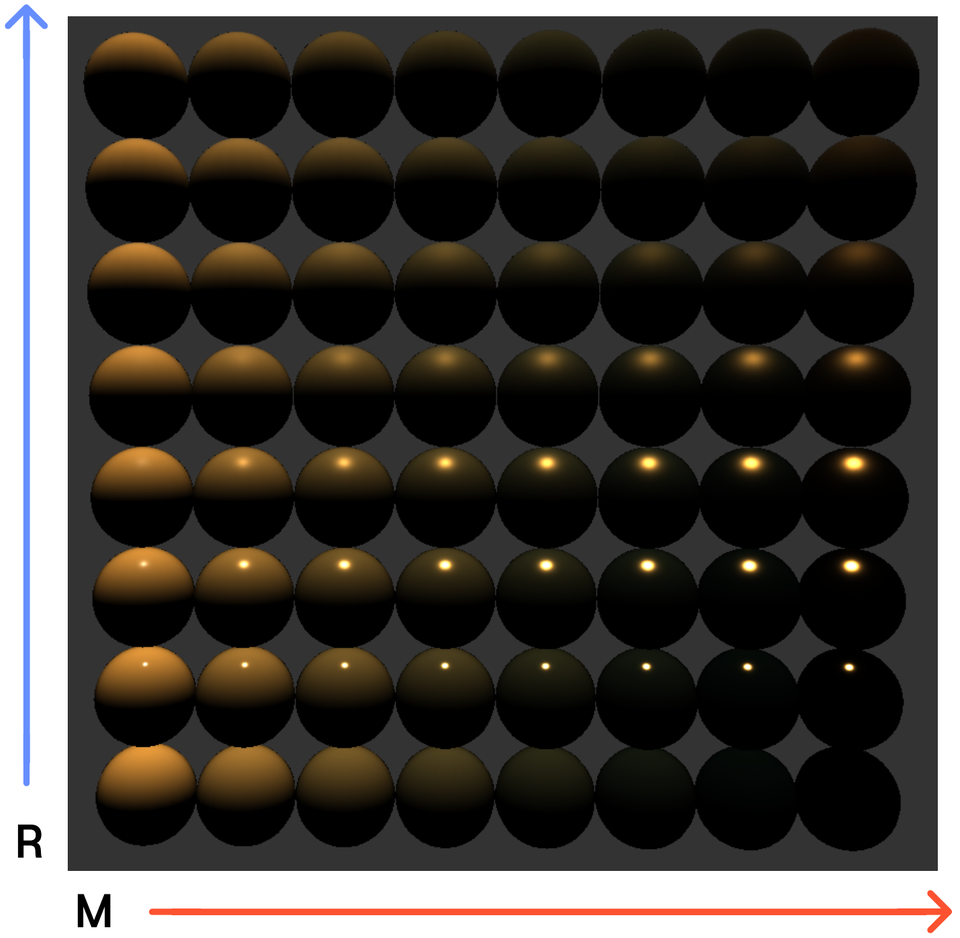


SUPER MARIO ODYSSEY

| PBR (Physical based Rendering)
| Depth Map그림자 & PCF
| Deferred Rendering
| 분리된 모델 본 연결
| Instancing
| Frustum Culling
| FSM & Capture기능
| 애니메이션 이벤트

PBR (Physical based Rendering)

- 에너지 보존법칙에 기반한 분석광 구현



Roughness, **Metalness**로
에너지 보존법칙 표현, 오브젝트의 재질 설정

Roughness가 높을수록
Specular ↓ ↓
Diffuse ↑ ↑
Albedo ↑ ↑

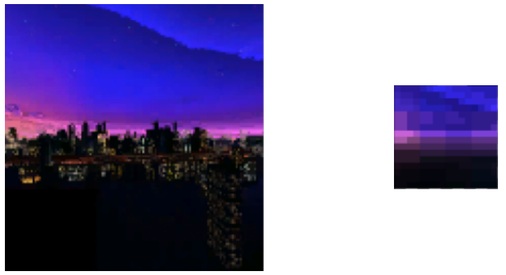
Metalness가 높을수록
Specular ↓ ↓
Diffuse ↑ ↑
Albedo ↑ ↑

SUPER MARIO ODYSSEY

| 컴포넌트 기반 오브젝트 구조
| PBR (Physical based Rendering)
| Depth Map그림자 & PCF
| Deferred Rendering
| 분리된 모델 본 연결
| Instancing
| Frustum Culling
| FSM & Capture기능
| 애니메이션 이벤트

PBR (Physical based Rendering)

- **IBL(Image Based Light)을 사용한 전역 조명**
  Irradiance Map으로 Diffuse, Specular Map으로 Specular 구현
  Specular Map을 Mipmapping 하여 표면의 거칠기 표현

Diffuse IBL	Specular IBL
 Irradiance Map	Roughness Mipmap level up →  Specular Map (Cubemap Texture)

SUPER MARIO ODYSSEY

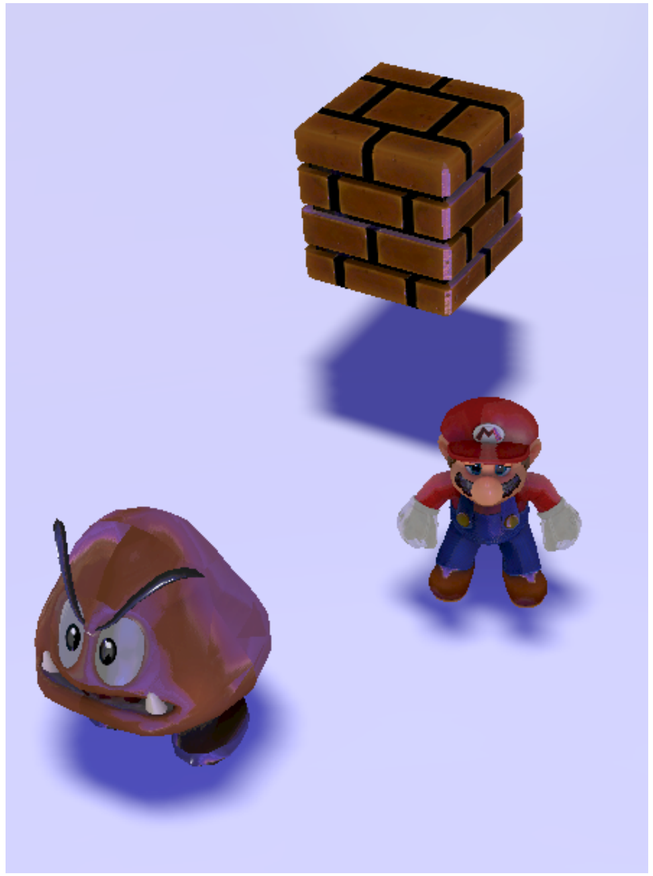
Depth Map그림자 & PCF

- Depth Map을 활용한 그림자 렌더링
- Depth를 비교하여 그림자 생성
- Shadow Acne 제거
  -> Bias로 해결
- 계단현상 없애기 위한 PCF
  (Percentage closer filtering)
  주변 텍셀을 여러 번 샘플링해 그림자 경계를 평균 처리

PCF 사용 X



PCF 사용 O



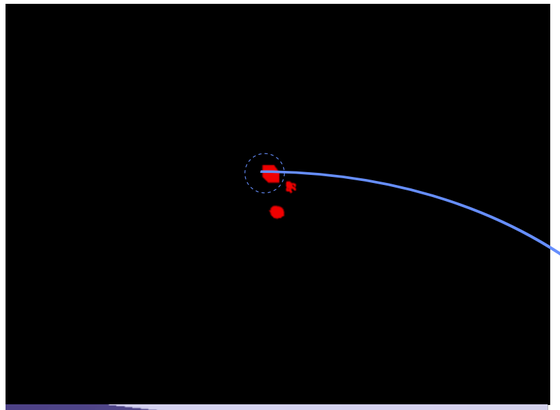
SUPER MARIO ODYSSEY

- PBR (Physical based Rendering)
- Depth Map그림자 & PCF
- Deferred Rendering
- 분리된 모델 본 연결
- Instancing
- Frustum Culling
- FSM & Capture기능
- 애니메이션 이벤트

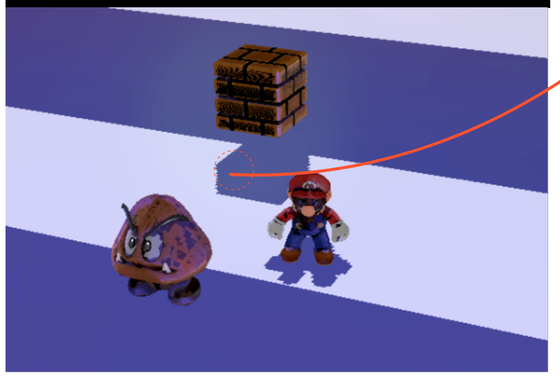
Depth Map그림자 & PCF

- Depth를 비교하여 그림자 생성
  카메라 시점의 깊이와 Light 시점의 깊이를 비교
  카메라 시점의 깊이가 더 깊으면 그림자로 표시

Depth Map



카메라 시점



깊이 비교

```
// Shadow map
float shadowFactor = 1.0;

// 1. Project posWorld to light screen
float4 lightScreen = mul(float4(posWorld, 1.0), light.view);
lightScreen = mul(float4(lightScreen.xyz, 1.0), light.projection);
lightScreen.xyz /= lightScreen.w;

// 2. 카메라(광원)에서 볼 때의 텍스춰 좌표 계산
float2 lightTexcoord =
float2(
    (lightScreen.x * 0.5) + 0.5f,
    -(lightScreen.y * 0.5) + 0.5f
);

// Baisc Shadow

// 3. 쉐도우맵에서 값 가져오기
float depth = shadowMap.Sample(clampSampler, lightTexcoord).r;

// 4. 가려져 있다면 그림자로 표시
if (depth + 0.00001 < lightScreen.z)
    shadowFactor = 0.0;
```

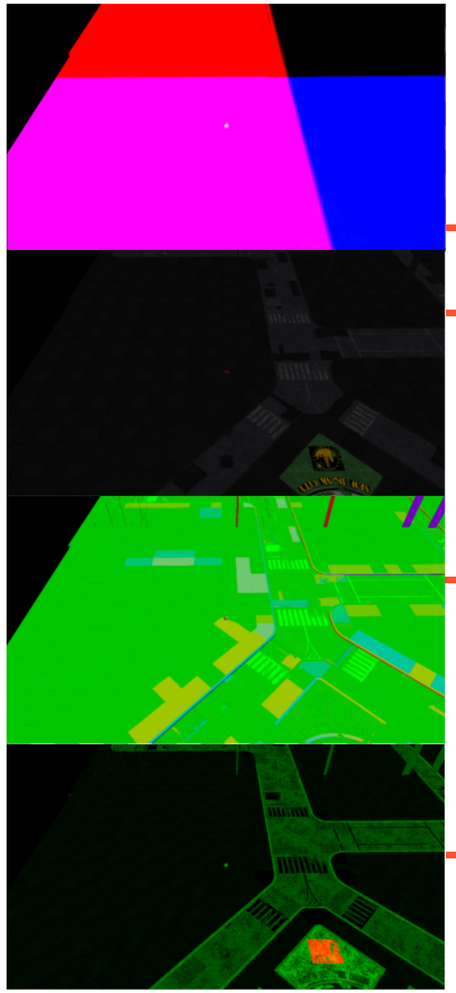
SUPER MARIO ODYSSEY

Deferred Rendering

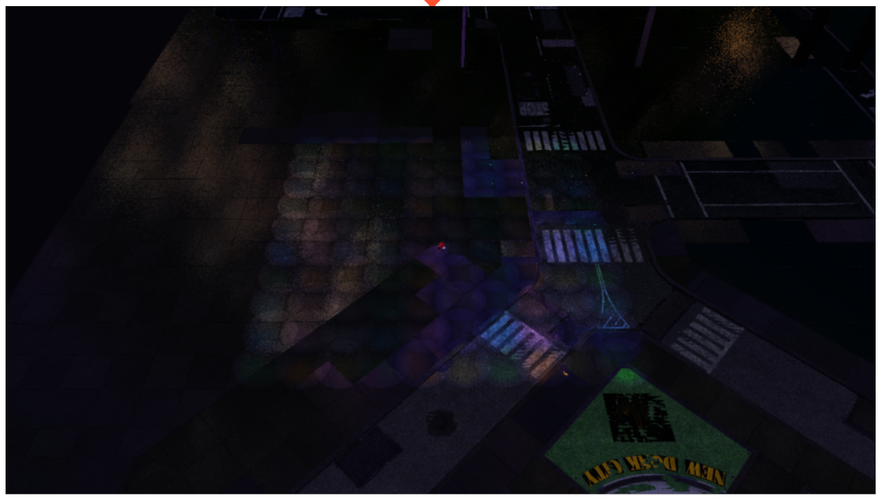
- 1차 패스(Geometry):
  씬 정보를 Buffer 4장에 기록
  Position / Albedo / Normal /
  Roughness / Metallic

- 2차 패스(Lighting):
  Buffer를 읽어 화면 픽셀 단위로
  라이팅 계산

- 광원이 많을수록 유리한 구조:
  픽셀당 한 번만 셰이딩
  (라이팅이 광원 수에 덜 민감)



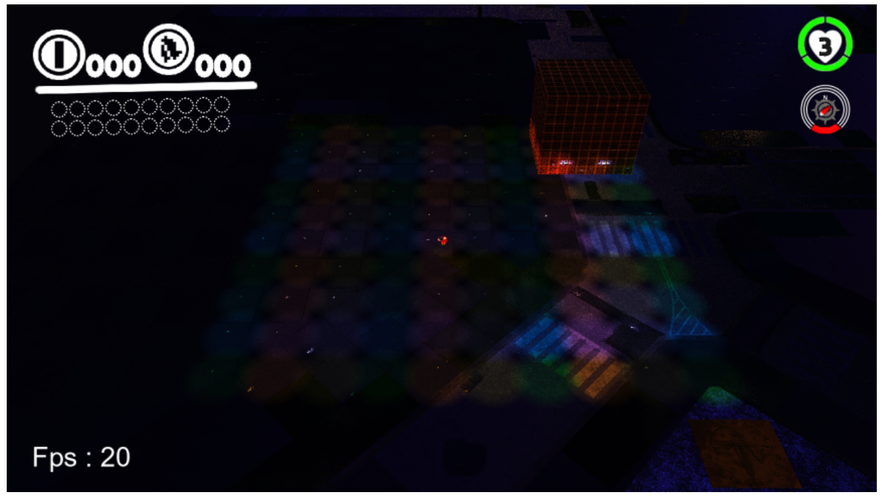
	CHANNELS			
RTNUM	R	G	B	a
RT1	POSITION			
RT2	ALBEDO			
RT3	NORMAL			
RT4	ROUGHNESS	METALLIC		



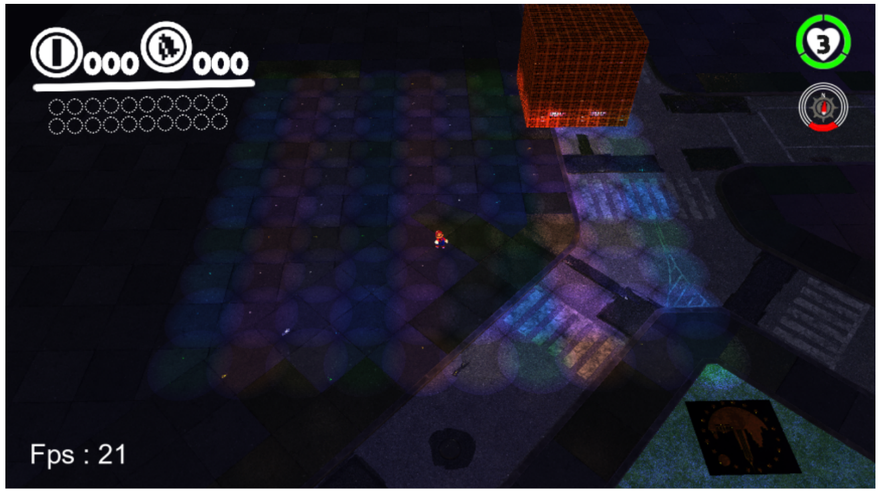
SUPER MARIO ODYSSEY

- PBR (Physical based Rendering)
- Depth Map그림자 & PCF
- Deferred Rendering
- 분리된 모델 본 연결
- Instancing
- Frustum Culling
- FSM & Capture기능
- 애니메이션 이벤트

Deferred Rendering



Forward Only



Deferred Only

Point Light 100개
오브젝트 1000개
프레임 차이 없었음
(디퍼드 렌더링이 무조건 이득이 아니라는 점 체득)

SUPER MARIO ODYSSEY

분리된 모델 본 연결

- 캐릭터를 MainBody + 파츠 (모자·콧수염·머리·눈)로 분리해 구성
- 파츠 본과 본체 본의 이름이 같으면 연결 (예: Armature_Hand.L)
- RootBone부터 재귀 순회, 부모 본의 누적 행렬 × 현재 본 로컬 행렬로 월드 행렬 완성
- 완성된 본에 파츠를 부착
  -> 본체 애니메이션을 그대로 따라감

Parts



MainBody





MainBody에 파츠들 연결

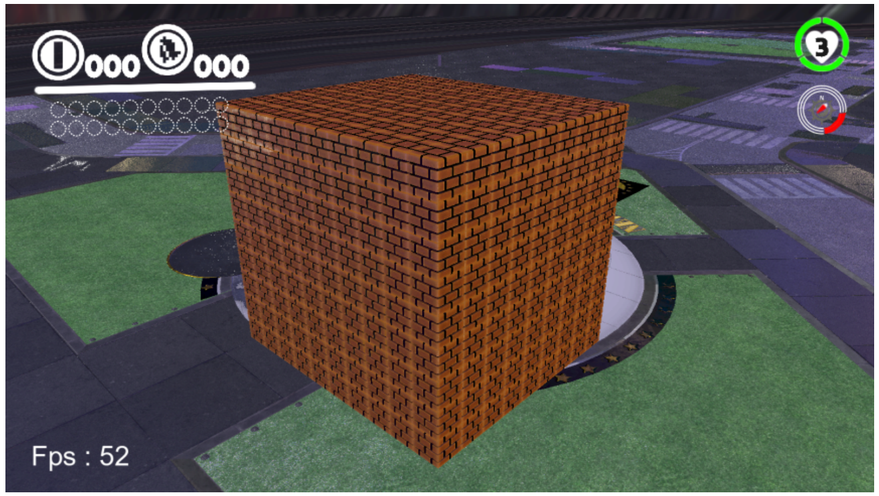


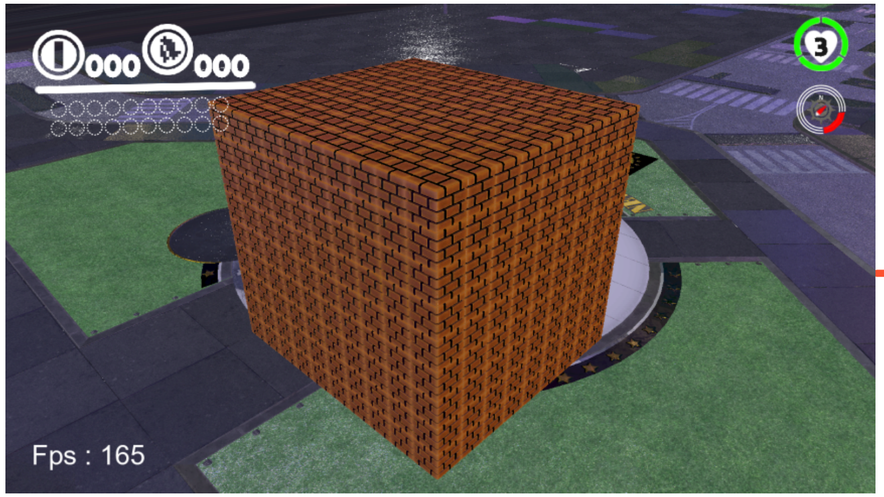
연결하지 않고 World 변환만 했을 때

SUPER MARIO ODYSSEY

Instancing

- 동일 메시·머티리얼을 한 번의 드로우콜로 대량 렌더링
- 인스턴스별 위치(World 행렬)를 InputLayout 데이터로 추가
- 메시 버퍼 + 인스턴스 버퍼를 함께 묶어 한 번의 드로우콜로 전부 렌더
- 메시는 공유하고, 인스턴스마다 위치만 다르게 적용





→ 인스턴싱 사용 :113 프레임 상승

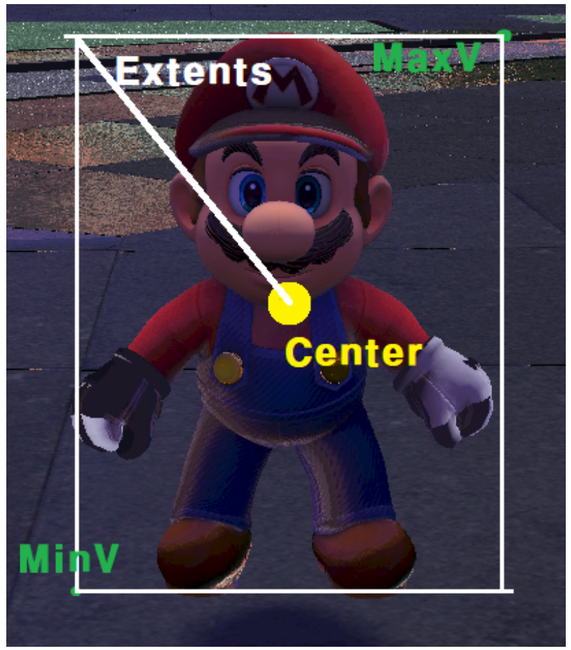
SUPER MARIO ODYSSEY

Frustum Culling

- 오브젝트마다 AABB를
  카메라 Frustum과 충돌 검사
- Frustum 밖이면 렌더링에서 제외
  (DirectXCollision의 AABB·Frustum
  사용)
- AABB는 모델의 최대/최소 정점으로
  Center·Extents 계산







→ Culling 사용 : 30 프레임 상승

SUPER MARIO ODYSSEY

애니메이션 이벤트

- 애니메이션 클립이 Start·Complete·End·KeyFrame 시점마다 std::function 콜백 보유
- 클립이 그 시점에 도달하면 등록된 콜백을 자동 실행
- 람다로 등록 → 애니메이션 전환 + 파티클/사운드를 정확한 타이밍에 트리거 (예: RunStart 완료 → Run 재생 + 파티클)

애니메이션 이벤트들

```
std::function<void()> mStartEvent;
std::function<void()> mCompleteEvent;
std::function<void()> mEndEvent;

std::map<UINT, std::function<void()>> mKeyFrameEvent;
```

```
void SetCompleteEvent(std::function<void()> inEvent) { mCompleteEvent = std::move(inEvent); }

cilp = animator->GetAnimationClip(L"RunStart");
if (cilp)
        cilp->SetCompleteEvent([animator, this]()
{
        animator->Play(L"Run");
        ParticleSystem* particle = GetParticle();
        if (particle)
        {
                particle->Play(L"Default");
        }
});
```

파티클 생성

